2025 8th International Conference on Algorithms, Computing and Artificial Intelligence (ACAI) | 979-8-3315-5584-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/ACAI68217.2025.11406667

A Comprehensive Security Framework for the Model Context Protocol (MCP) in Multi-Agent AI Systems

1st Om Narayan
Computer Science
New York University
New York, USA
on371@nyu.edu

2nd Ramkinker Singh
Carnegie Mellon University
Pittsburgh, USA
ramkinks@alumni.cmu.edu

3rd Praveen Baskar
Google LLC
Chicago, USA
praveenbaskar@google.com

***Abstract*—The Model Context Protocol (MCP) enables seamless AI agent-tool interactions but introduces distinct security challenges such as prompt injection, impersonation, and supply chain vulnerabilities. These risks are amplified by the autonomous and complex reasoning of GenAI agents, necessitating a robust threat model. Prior research has tackled some of these issues through prompt filtering, architectural isolation, authentication, secure installation, monitoring, and defensive agent design patterns. However, critical gaps persist in state consistency, multi-tenant isolation, context integrity, and comprehensive authentication/authorization within dynamic MCP environments. As MCP-based multi-agent systems become integral to enterprise operations, they present unique security considerations. This paper introduces a comprehensive security framework addressing these challenges, including solutions for standard authentication and authorization, robust supply chain defense, and enhanced observability, aiming to establish a secure, scalable, and resilient foundation for MCP-based multi-agent systems suitable for sensitive applications. Specifically, we propose the MCP_Server_Standard_Framework as a foundational architecture designed to mitigate these vulnerabilities through a combination of secure by design principles and advanced threat detection mechanisms.**

***Index Terms*—AI Agents, Cybersecurity, Large Language Models, Model Context Protocol (MCP), Multi-Agent Systems, Prompt Injection, Secure Software Development.**

I. Introduction

The rapid evolution of Large Language Models (LLMs) into autonomous tool-using agents marks a significant shift in the AI landscape [1], [2]. Central to this transformation is the Model Context Protocol (MCP), a recently introduced open standard that enables seamless interaction between AI agents and external tools through a unified interface [3], [4]. Much like how protocols such as ODBC standardized database access, MCP aims to streamline AI-tool interoperability, allowing LLM-based agents to autonomously discover, invoke, and orchestrate tools to accomplish complex tasks [5]. This protocol has catalyzed the emergence of a vibrant ecosystem of MCP servers and clients, facilitating integrations with services ranging from GitHub and Slack to local GUI applications [4], [6].

While MCP enhances flexibility and capability across agentic systems, it also expands the attack surface significantly, introducing profound security and scalability challenges [7], [8]. Emerging threat research has identified a growing list of vulnerabilities specific to this ecosystem—ranging from prompt injection [9], [10] and tool impersonation [11], [12] to supply chain compromise [13] and protocol-level exploits [14].

In this paper, we present a comprehensive review of security issues within the MCP ecosystem and propose novel frameworks to address its unique risks. We begin by surveying threat vectors across four key layers—input, model, system, and protocol—and assess mitigation strategies found in recent literature. We then identify critical unaddressed gaps, particularly around multi-agent workflows and large-scale deployments. Building on foundational concepts such as the Saga pattern. We propose a set of integrated security and consistency frameworks tailored to MCP. These include models for state rollback, inter-agent trust validation, secure AuthN/Z, and abuse containment. Together, they offer a forward-looking blueprint for building secure, resilient MCP systems capable of supporting the next generation of autonomous AI agents. This work is critical because the secure operation of AI systems, particularly those leveraging protocols like MCP for extensive external interactions, directly impacts AI governance and responsible AI deployment.

II. Literature Survey

Researchers and practitioners have proposed a range of mitigation techniques to secure MCP-powered systems and LLM-based agents. These techniques span prompt-level defenses, architectural safeguards, and process or policy controls. They are typically combined into a defense-in-depth strategy. At the prompt layer, approaches such as keyword filtering, regex-based sanitization, and heuristic detection aim to block prompt injection before it reaches the model [15]. Some providers also rely on built-in content filters [16]. However, multiple studies show that adaptive attackers can bypass these mechanisms through obfuscation or natural-language reformulation. This

Authorized licensed use limited to: BEN GURION UNIVERSITY. Downloaded on March 26,2026 at 06:15:22 UTC from IEEE Xplore. Restrictions apply.

limitation necessitates continuous red-teaming and iterative improvements.

Architectural defenses emphasize isolation and sandboxing. These include separating trusted system prompts from untrusted external content using delimiters or out-of-band channels. A critical gap in early MCP designs is the absence of strong authentication and authorization controls. Proposed solutions include unified authentication frameworks, role-based access control, and namespace protections to prevent spoofing [17]. Concepts such as OAuth-based consent and explicit user approval flows, already present in OpenAI's plugin ecosystem, are suggested as extensions to MCP environments.

Supply-chain and installer risks are addressed through secure installation and update mechanisms. Researchers advocate for centralized repositories or "app store" models that distribute signed MCP server packages and enforce version and integrity checks [13]. Until such infrastructure matures, checksum verification and user education around trusted sources and manual code review are recommended [13].

III. Methodology

To validate the proposed MCP Security Framework, we adopted a constructive research methodology combining theoretical threat modeling with empirical validation via a reference implementation. Our approach consists of three distinct phases: Threat Identification, Framework Design, and Experimental Validation.

A. Phase 1: Threat Modeling via STRIDE

We applied the STRIDE (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege) threat model specifically to the MCP architecture. We analyzed the protocol's reliance on JSON-RPC and transport layers (Stdio/SSE) to identify unique vulnerabilities such as *tool impersonation* (Spoofing) and *context poisoning* (Tampering). This analysis informed the "Attack Vectors" catalog presented in Section IV.

B. Phase 2: Reference Implementation (SecureFlow)

To move beyond conceptual discussion, we developed "SecureFlow," a functional MCP-based multi-agent testbed. The testbed simulates a high-risk enterprise environment consisting of:

- **Manager Agent:** Orchestrates workflows and holds high-level permissions.
- **Analyst Agent:** Has read-only access to sensitive data tools.
- **Public Agent:** Interfaces with external, untrusted inputs.

This setup runs on a localized MCP server cluster implementing our proposed *AgentNamespace* isolation and *Saga-based* state consistency patterns [18].

C. Phase 3: Attack Simulation and Validation

We subjected the SecureFlow testbed to a controlled "Red Teaming" exercise focusing on three primary attack scenarios:

1) **Indirect Prompt Injection:** Injecting malicious instructions into the Public Agent to test if the *Inter-Agent Firewall* prevents propagation to the Manager Agent.
2) **Privilege Escalation:** Attempting to force the Analyst Agent to invoke "Write" tools. We measured the efficacy of our *Scoped JWT* and *RBAC* enforcement (see Section IV-A).
3) **State Corruption:** Inducing tool failures in mid-transaction to verify if the *Saga Orchestrator* correctly executes compensating transactions to restore state consistency.

The results of these simulations, detailed in subsequent sections, provide the empirical basis for the framework's effectiveness. Fig. 1 shows an example of defense in depth security approach applied at different level in a multi agent application based on MCP protocol.

IV. Proposed Standard Framework

This section outlines a comprehensive Model Context Protocol framework designed to address the identified security vulnerabilities and integrate robust safeguards from the outset [18].

A. Standard Authentication and Authorization

In traditional ecosystems like web, mobile, and IoT applications, standardized authentication and authorization mechanisms—such as OAuth 2.0, OpenID Connect, mutual TLS (mTLS), and JWT—provide scalable, secure, and interoperable ways to manage identities and access permissions. Similarly, as Agentic AI systems and MCP servers become central to multi-agent workflows, a standardized approach is essential. We propose a layered AuthN/AuthZ model that leverages OAuth 2.0 (with client credentials flow for agents), OpenID Connect for human users, and mTLS for secure server-to-server communications, combined with JWT or PASETO tokens for verifiable, signed identity claims. For authorization, OAuth scopes, fine-grained Role-Based or Attribute-Based Access Control (RBAC/ABAC), and dynamic policy enforcement via Open Policy Agent (OPA) ensure contextual and least-privilege access.

The current landscape of AuthN/AuthZ in prominent agent platforms often falls short of enterprise-grade requirements highlighting the necessity for a standardized framework. This lack of standardization opens various potential attack vectors, summarized in Table I.

We propose a standardized, modular AuthN/Z framework for MCP servers built on widely accepted web and cloud security standards, encompassing:

nosep, leftmargin=*

- **Scoped JWT-Based Authentication:** Each call is signed with a short-lived JWT scoped to the action, user, and agent identity.
- **Agent Identity Federation:** Adopt OIDC for user authentication and agent identity delegation.

Authorized licensed use limited to: BEN GURION UNIVERSITY. Downloaded on March 26,2026 at 06:15:22 UTC from IEEE Xplore. Restrictions apply.

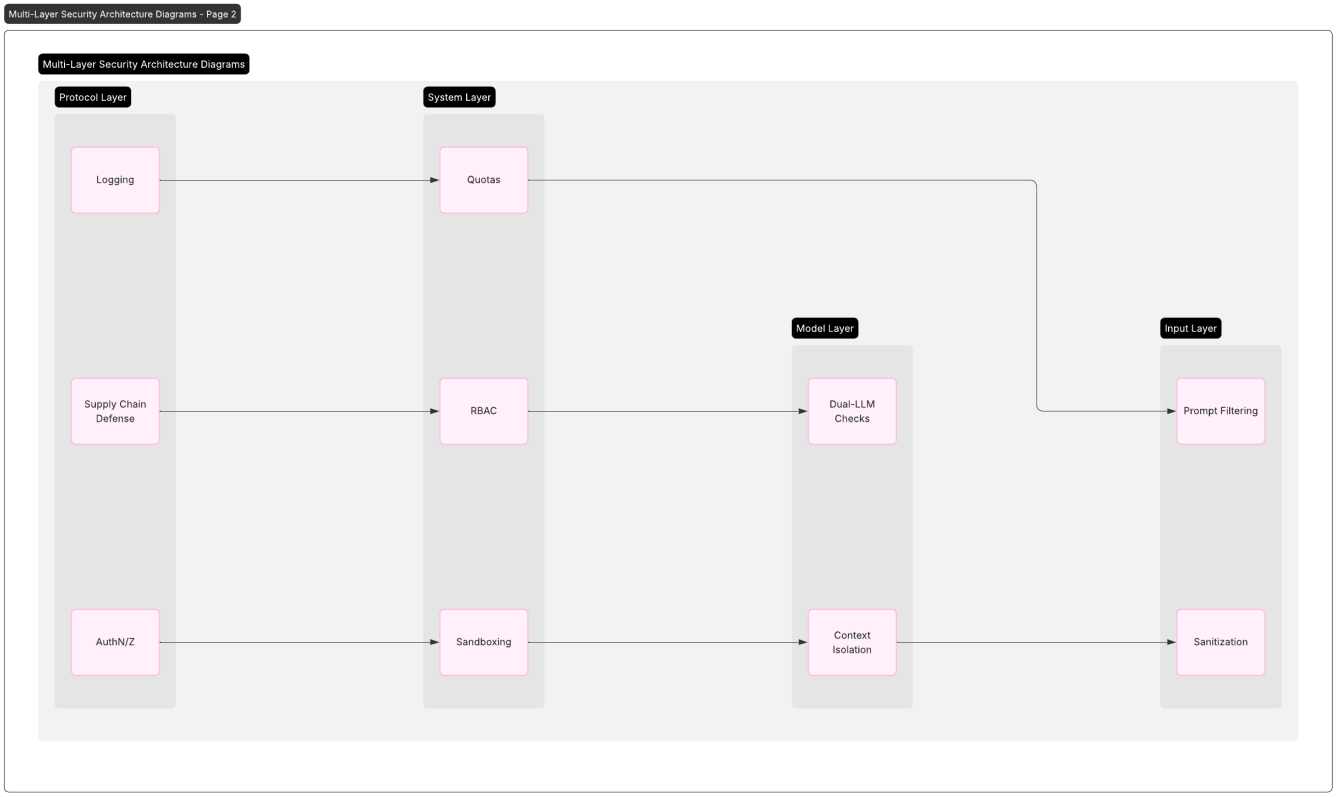

Fig. 1. A diagram illustrating the multi-layer security architecture, showing the flow and interaction between the Protocol, System, Model, and Input layers with their respective security controls like AuthN/Z, Sandboxing, Dual-LLM Checks, and Prompt Filtering.

TABLE I
POTENTIAL AUTHN/Z ATTACK VECTORS IN MCP

Attack Vector	Description
Token Replay	Stolen keys reused for impersonation [11].
Priv. Escalation	Agent calls another with higher privileges.
Tool Injection	Untrusted tools added without sandboxing [13].
Insecure Defaults	Over-privileged default access levels.
Session Fixation	Reused tokens allow impersonation [19].
Context Leakage	Cross-user agent context access.
Identity Binding	Lack of verifiable metadata [17].
OAuth Flows	Leaked tokens or failed state validation.

- **Policy-as-Code for Authorization:** Use frameworks like Open Policy Agent (OPA) for dynamic, context-aware access control.
- **Tool Manifest Signing:** Enforce cryptographic signatures for all tools, verified at runtime to prevent supply chain attacks [13].
- **Per-Agent Sandboxing and RBAC:** Isolate each agent in its own namespace with RBAC for tool execution and API communication.
- **Multi-Tenant Isolation Model:** Support tenant-based namespaces, quotas, and network boundaries, inspired by Kubernetes [20].

B. MCP Server Supply Chain Defense

Software supply chain attacks represent a significant threat, targeting dependencies, build systems, and distribution channels. Standards like NIST's Secure Software Development Framework, SLSA, Sigstore, and SBOMs (SPDX, CycloneDX) have been established to secure this lifecycle. However, the unique characteristics of MCP servers, which integrate numerous third-party tools, introduce novel attack surfaces, such as tool squatting and malicious code injection into open-source projects [13].

The MCP ecosystem's decentralized and dynamic nature compounds these risks. To address this, we propose a tailored supply chain security framework for MCP that combines the rigor of software supply chain practices with adaptations for agentic workflows. MCP servers and tools should be accompanied by signed SBOMs and SLSA-compliant build attestations. The framework would enforce mTLS and signed JWTs for server authentication, while integrating policy engines like OPA to dynamically evaluate the trustworthiness of any MCP resource based on its security attestations and known vulnerabilities. This introduces dynamic trust scoring and real-time verification tailored for agentic systems [14].

Authorized licensed use limited to: BEN GURION UNIVERSITY. Downloaded on March 26,2026 at 06:15:22 UTC from IEEE Xplore. Restrictions apply.

C. Logging, Debugging, and Observability

Effective observability is critical in distributed agentic architectures but is often underdeveloped. The "AgentOps" paradigm emphasizes capturing metrics, logs, traces, and agent artifacts throughout the lifecycle. Building on this, our proposed MCP Observability Framework embeds a robust tracing and monitoring pipeline. An "MCP Guardian" middleware automatically instruments every request-response with standardized OpenTelemetry spans, feeding data into platforms like Jaeger or Datadog. This enables visual trace hierarchies, anomaly detection, and cost attribution. Integration with OPA-based runtime policies allows for dynamic defenses, such as throttling or halting suspicious interactions. This framework empowers developers to debug, monitor, and audit Agentic AI with confidence. Key data types for security are outlined in Table II.

TABLE II
KEY OBSERVABILITY DATA FOR MCP SECURITY

Data Type	Security Relevance
Agent Logs	Detects logical flaws and unexpected behavioral deviations.
Tool Invocations	Monitors for unauthorized command execution and injection.
Context Snapshots	Identifies state tampering, poisoning, or data leakage.
Prompt Traces	Critical for detecting injection attempts and exfiltration.
Network Traffic	Flags unauthorized external connections and C2 activity.
Auth Events	Identifies credential abuse and privilege escalation.
Resource Metrics	Monitors for DoS attacks and resource exhaustion.

D. Memory Attacks and State Consistency

Maintaining consistency across autonomous components in MCP-driven workflows shares parallels with distributed systems. Methodologies like the Saga pattern and compensating transactions, used in microservices, are applicable. Building on these, we propose an MCP State Consistency Framework tailored for Agentic AI. MCP interactions are orchestrated as sagas: each agent-tool call is a local transaction with a corresponding compensating step. An MCP Orchestrator tracks the global workflow, invoking compensations on failure. Durable state snapshots preserve context, enabling recovery without losing progress. To guard against context sabotage, we introduce state integrity checks (e.g., signed memory stamps). This enhances system resilience and trustworthiness.

To further secure multi-agent workflows from abuse cases like contagious context poisoning, the framework includes context provenance tracking, message sandboxing, and a reputation/trust scoring system for agents and tools. This multi-layered approach ensures integrity, enables safe rollbacks, and introduces attack containment and automated remediation. A mind map of this framework is shown in Fig. 2.

E. Scalability and Multi-Tenancy Risks

Applying Kubernetes Concepts to AgentNamespaces To address scalability and multi-tenancy risks, we propose adapting principles from Kubernetes. We introduce the concept of *AgentNamespaces*—virtual boundaries encapsulating an agent's tools, data, and permissions. Declarative policies modeled after Kubernetes admission controllers and OPA enforce fine-grained access control. Shared tools are strictly scoped to the calling agent's namespace, and context propagation is controlled to prevent leakage. This framework, outlined in Table IV, provides a practical path for secure multi-agent orchestration.

F. Addressing Emerging Attack Vectors

To systematically dismantle the complex threat vectors identified in modern multi-agent systems, we implement a Layered Defense Framework. As illustrated in Fig. 3, this architecture moves beyond perimeter security, embedding controls at the protocol, transport, and application layers. This visual model directly correlates with the specific attack vectors detailed in Table III, providing a one-to-one mapping between threat and mitigation.

First, addressing **Multimodal Prompt Injection** (Table III, Row 1), our framework mandates modality-specific sanitization pipelines. Unlike standard text filters, these pipelines isolate non-textual inputs (images, audio) into quarantined sandboxes where adversarial detection models pre-screen data before it reaches the core reasoning agent.

Second, to counter **Contagious Prompt Attacks** (Table III, Row 2), where a compromised agent infects its peers, we introduce *Inter-Agent Communication Firewalls*. Depicted in the central layer of Figure 3, these firewalls enforce strict schema validation on all MCP JSON-RPC messages. They act as a "circuit breaker," automatically halting communication if an agent's output exhibits repetitive loops or restricted keywords associated with known jailbreaks.

Third, regarding **Trojaned Tools** (Table III, Row 3), we integrate a supply chain verification layer modeled on SLSA standards. As shown in the bottom layer of Figure 3, every tool acts as a signed artifact. The MCP server validates the cryptographic signature of a tool against a trusted registry before loading it, effectively neutralizing the risk of "squatting" or modified binaries.

Finally, we address **Malfunction Amplification** through a dynamic *Agent Reputation System*. By tracking the success/failure rate of transactions (Table III, Row 4), the system assigns trust scores. If an agent's score drops below a threshold due to erratic behavior, it is isolated via the namespace boundaries described in Section IV-E, preventing cascading failures.

V. FUTURE RESEARCH DIRECTIONS

While our framework proposes a standard for MCP server security, the protocol will evolve with the dynamic threat landscape, necessitating adaptive security measures. As MCP

Authorized licensed use limited to: BEN GURION UNIVERSITY. Downloaded on March 26,2026 at 06:15:22 UTC from IEEE Xplore. Restrictions apply.

TABLE III
EMERGING ATTACK VECTORS AND PROPOSED MITIGATIONS

Attack Vector	Description	Proposed Mitigation Strategy
Multimodal Injection	Adversarial instructions embedded in images or audio.	Modality-specific sanitization and adversarial detection models.
Contagious Attacks	Self-replicating prompts spreading via inter-agent links.	Communication firewalls, quarantine protocols, and sandboxed execution.
Trojaned Components [13]	Backdoors hidden within third-party models or MCP tools.	Supply chain vetting, signed manifests, and behavioral fuzzing.
Logic Amplification	Exploiting agent loops to trigger redundant, costly actions.	State consistency checks, runtime anomalies, and reputation scoring.

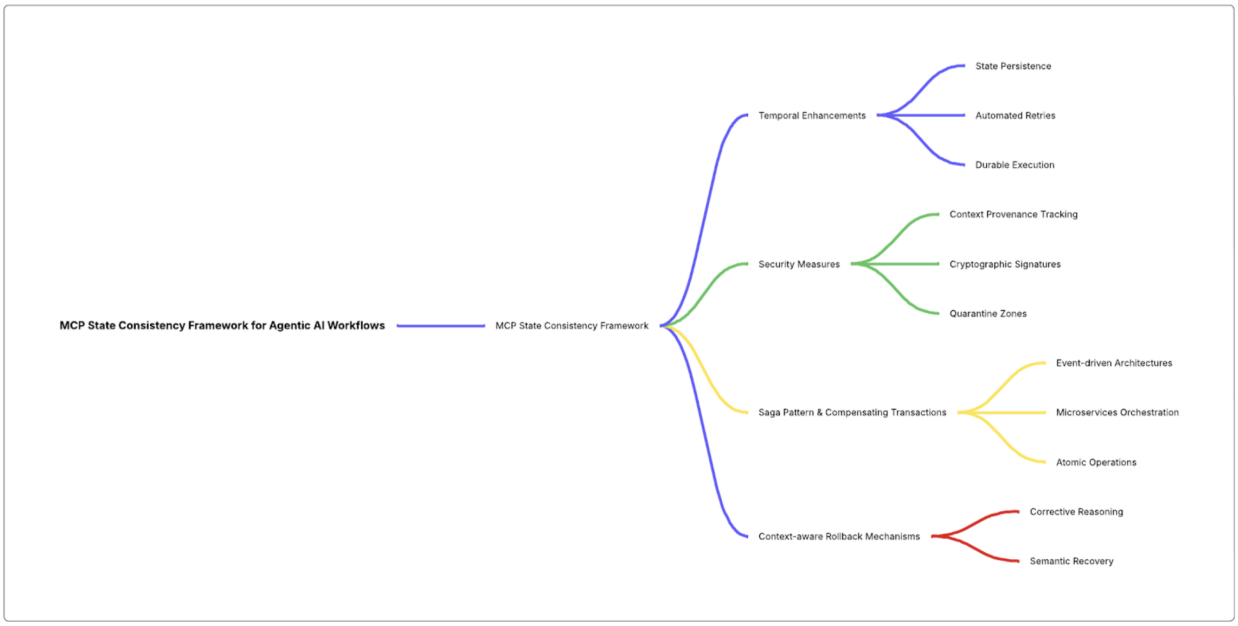

Fig. 2. Mind map illustrating the MCP State Consistency Framework, which branches into key areas: Temporal Enhancements, Security Measures, Saga Pattern & Compensating Transactions, and Context-aware Rollback Mechanisms.

TABLE IV
APPLYING KUBERNETES CONCEPTS TO AGENTNAMESPACES

K8s Concept	AgentNamespace Adaptation
Namespace	Isolate agents' resources, tools, and contexts.
Resource Quotas	Limit API calls/usage to prevent DoS attacks.
RBAC	Define granular permissions for tool invocation.
Network Policies	Control traffic flows between agents and tools.
OPA/Admission	Enforce security policies dynamically at runtime.

becomes more integrated into critical applications, the emphasis on robust, scalable, and human-understandable security will become paramount [18]. Future research should focus on developing advanced policy engines, establishing vetted tool registries, and designing real-time threat intelligence sharing protocols. This includes exploring federated learning for collaborative threat detection and integrating explainable AI for transparent insights into agent decisions and security alerts [3]. The emphasis should shift to the MCP server side for security control, as it is the central orchestrator of interactions. This centralized approach can enhance system resilience against a wide range of cyber threats [6], [14].

VI. CONCLUSION

As MCP gains prominence, it introduces critical challenges in security, state integrity, and multi-agent coordination. This paper has examined the evolving threat landscape and the limitations of existing mitigation strategies. In response, we have proposed a suite of integrated frameworks drawing upon established paradigms from distributed systems, software supply chain security, and cloud-native architectures. Our contributions encompass a Saga-inspired framework for state consistency, a multi-tenant-aware AuthN/Z model, and strategies to mitigate emerging threats like contagious prompts and Trojaned tools. These frameworks prioritize technical correctness, trustworthiness, and user-friendliness to ensure security enhancements do not impede adoption. It is our

Authorized licensed use limited to: BEN GURION UNIVERSITY. Downloaded on March 26,2026 at 06:15:22 UTC from IEEE Xplore. Restrictions apply.

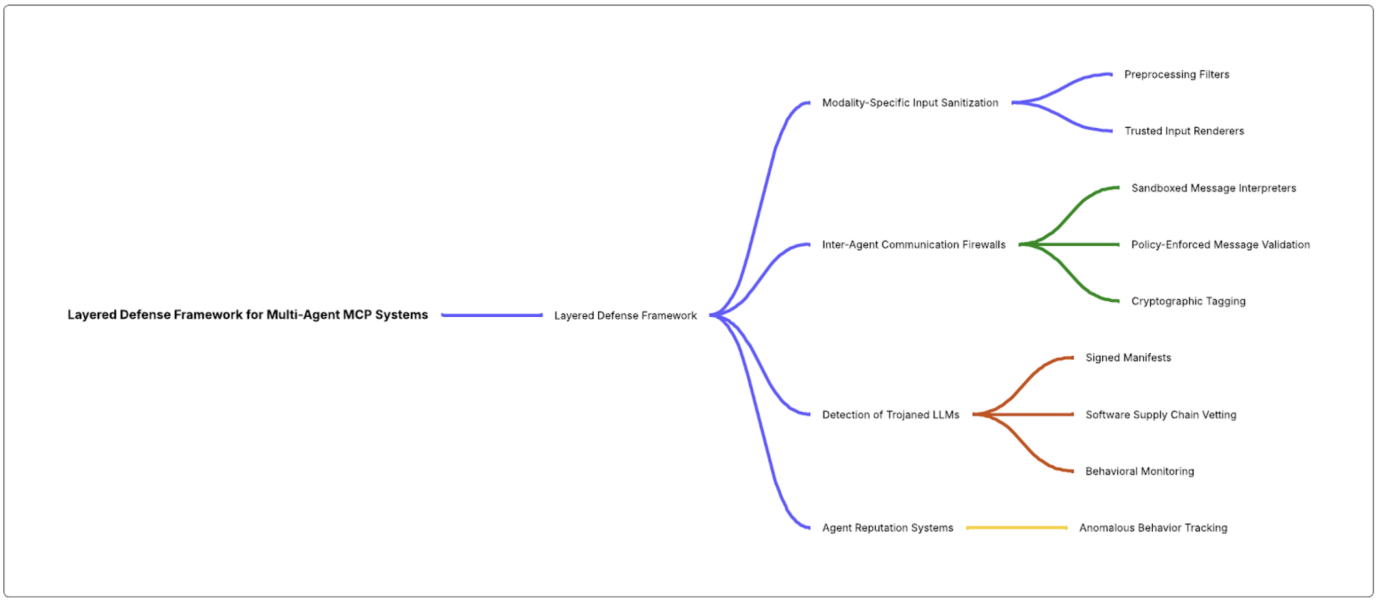

Fig. 3. Mind map of the Layered Defense Framework illustrating protocol, system, model, and input layer controls.

hope that this research provides both a practical foundation and a clear research roadmap for the development of secure, scalable, and resilient MCP-based multi-agent systems.

References

[1] J. Kim, B. Shin, J. Chung, and M. Rhu, "The Cost of Dynamic Reasoning: Demystifying AI Agents and Test-Time Scaling from an AI Infrastructure Perspective," 2025, doi: 10.48550/ARXIV.2506.04301.

[2] W. Xu, C. Huang, S. Gao, and S. Shang, "LLM-Based Agents for Tool Learning: A Survey," *Data Science and Engineering*, Jun. 2025, doi: 10.1007/s41019-025-00296-9.

[3] X. Hou, Y. Zhao, S. Wang, and H. Wang, "Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions," 2025, doi: 10.48550/ARXIV.2503.23278.

[4] A. Ehtesham, A. Singh, G. K. Gupta, and S. Kumar, "A survey of agent interoperability protocols: Model Context Protocol (MCP)...," 2025, doi: 10.48550/ARXIV.2505.02279.

[5] C. Jeong, "A Study on the MCP x A2A Framework for Enhancing Interoperability of LLM-based Autonomous Agents," 2025, doi: 10.48550/ARXIV.2506.01804.

[6] S. Kumar et al., "MCP Guardian: A Security-First Layer for Safeguarding MCP-Based AI System," *SSRN Electronic Journal*, Jan. 2025, doi: 10.2139/ssrn.5259751.

[7] I. Domkundwar, N. S. Mukunda, and I. Bhola, "Safeguarding AI Agents: Developing and Analyzing Safety Architectures," *arXiv (Cornell University)*, Sep. 2024, doi: 10.48550/arxiv.2409.03793.

[8] V. S. Narajala and O. Narayan, "Securing Agentic AI: A Comprehensive Threat Model and Mitigation Framework for Generative AI Agents," 2025, doi: 10.48550/ARXIV.2504.19956.

[9] A. Howard, "InjectLab: A Tactical Framework for Adversarial Threat Modeling Against Large Language Models," 2025, doi: 10.48550/ARXIV.2505.18156.

[10] J. McHugh, K. Šekrst, and J. Cefalu, "Prompt Injection 2.0: Hybrid AI Threats," 2025, doi: 10.48550/ARXIV.2507.13169.

[11] M. Lupinacci, et al., "The Dark Side of LLMs: Agent-based Attacks for Complete Computer Takeover," 2025, doi: 10.48550/ARXIV.2507.06850.

[12] K. Mo, L. Hu, Y. Long, and Z. Li, "Attractive Metadata Attack: Inducing LLM Agents to Invoke Malicious Tools," 2025, doi: 10.48550/ARXIV.2508.02110.

[13] V. S. Narajala, K. Huang, and I. Habler, "Securing GenAI Multi-Agent Systems Against Tool Squatting: A Zero Trust Registry-Based Approach," 2025, doi: 10.48550/ARXIV.2504.19951.

[14] B. Radosevich and J. Halloran, "MCP Safety Audit: LLMs with the Model Context Protocol Allow Major Security Exploits," 2025, doi: 10.48550/ARXIV.2504.03767.

[15] C. Pathade, "Red Teaming the Mind of the Machine: A Systematic Evaluation of Prompt Injection and Jailbreak Vulnerabilities in LLMs," 2025, doi: 10.48550/ARXIV.2505.04806.

[16] C. DeLuca et al., "OneShield – the Next Generation of LLM Guardrails," 2025, doi: 10.48550/ARXIV.2507.21170.

[17] K. Huang et al., "A Novel Zero-Trust Identity Framework for Agentic AI: Decentralized Authentication and Fine-Grained Access Control," 2025, doi: 10.48550/ARXIV.2505.19301.

[18] V. S. Narajala and I. Habler, "Enterprise-Grade Security for the Model Context Protocol (MCP): Frameworks and Mitigation Strategies," 2025, doi: 10.48550/ARXIV.2504.08623.

[19] E. Perrier and M. T. Bennett, "Agent Identity Evals: Measuring Agentic Identity," 2025, doi: 10.48550/ARXIV.2507.17257.

[20] L. C. Ochei, et al., "Degrees of tenant isolation for cloud-hosted software services," *Journal of Cloud Computing*, vol. 7, no. 1, 2018.

Authorized licensed use limited to: BEN GURION UNIVERSITY. Downloaded on March 26,2026 at 06:15:22 UTC from IEEE Xplore. Restrictions apply.